DWAANI AI RECRUITMENT PLATFORM

Autonomous AI Technical Screening & Assessment Suite

COMPLETE FLOW & ARCHITECTURAL SPECIFICATION

Document Meta Information

SYSTEM SPECIFICATION ATTRIBUTES

x95 Product Designation: Dwaani (InterviewAI / DwaniAI)
x95 Backend Architecture: Django 5.0, Django Channels (ASGI), Daphne, Redis Backplane
x95 Frontend Application: React 19, Vite 8, Tailwind CSS, Lucide Icons, CapacitorJS Mobile
x95 AI Engine Pipeline: Groq (Llama-3-70B / gpt-oss-20b), Whisper-large-v3, Deepgram Nova-3
x95 Database Architecture: PostgreSQL (Neon.tech) Relational Schema, In-Memory Redis Layer
x95 Document Scope: Test Creation, Test Evaluation, Proctoring Engine & Application Structure
x95 Release Status: Enterprise Edition 2.0 (Confidential & Proprietary)

Executive Architecture Summary

Dwaani is an autonomous recruitment and technical screening suite built to eliminate the massive bottlenecks of technical recruitment. It unifies resume extraction, semantic ATS alignment matching, voice-first adaptive technical interviews via WebSockets, 3-dimensional answer evaluation (Relevance, Accuracy, Clarity), multi-layered proctoring, and instant executive scorecards with live INR cost tracking.

SIX CORE SYSTEM PILLARS DETAILED IN THIS SPECIFICATION

x95 1. Test & Assessment Creation Flow: Job description ingestion, ATS matching, candidate invitation
x95 2. Candidate Onboarding: 6-stage hardware check (AudioContext singleton, screen share, network RTT)
x95 3. Adaptive Interview Chamber: Voice-first WebSockets, STT streaming, dynamic difficulty modulation
x95 4. Parallel Test Evaluation Engine: 3D scoring (Relevance, Accuracy, Clarity) via ThreadPoolExecutor
x95 5. Multi-Layered Proctoring: Tab-switch listeners, dual-frame canvas blending, anomaly thresholding
x95 6. Report Compilation & INR Telemetry: Candidate review, AI summary, PDF scorecard, token cost in INR

1. Test & Assessment Creation Flow

The Test Creation phase enables recruiters to establish job requisitions, upload candidate resumes, execute real-time semantic ATS alignment evaluations, configure interview difficulty parameters, and issue secure expirable credentials to applicants.

1.1 Document Extraction & Semantic ATS Matching

- Document Ingestion: Recruiter uploads Job Description (JD) and Candidate Resume (PDF/DOCX) via Jobs.jsx.
- Text Pre-Processing: DocumentExtractor (document_utils.py) strips malformed bytes, control chars, font tables.
- Groq ATS Engine: Sends sanitized JD & resume text to GroqAIService.process_screening() running Llama-3.
- Semantic Matching: Matches core tech stack (40%), experience depth (30%), architecture (20%), education (10%).
- Extraction Output: Yields verified skills array, candidate email/name, highlights, and ATS Score (0-100%).

1.2 Candidate Account & Cryptographic Link Provisioning

Upon recruiter approval of the extracted ATS match, the InviteCandidateView API endpoint orchestrates the candidate credentialing and session registration pipeline:

Component
Session Token
link1 (Candidate)
link2 (Recruiter)
Temp Password
Email Notification
In-App Alerts

TEST CREATION DATA FLOW SEQUENCE

x95 Recruiter uploads JD & Resume x97> POST /screening/process/ x97> DocumentExtractor x97> Groq ATS
x95 Groq returns ATS Score (0-100%) & Profile JSON x97> Recruiter sets num_questions (default: 5) & Schedule
x95 Recruiter clicks 'Invite' x97> POST /api/v1/interviews/invite/ x97> UUID session_token & temp password generated
x95 Interview model persisted with num_questions=5, ats_score, skills x97> SMTP sends HTML email with magic link

2. Candidate Onboarding & Pre-Flight System Check

Before candidates are permitted into the active assessment chamber, they must complete a rigorous 6-stage hardware, biometric, and network verification flow implemented in SystemCheck.jsx. This prevents technical disruptions and verifies hardware readiness before the interview starts.

Stage #
Stage 1
Stage 2
Stage 3
Stage 4
Stage 5
Stage 6

2.1 AudioContext Singleton Architecture

Standard web applications often fail voice assessments due to WebRTC microphone stream drops when switching routes. Dwaani solves this with a persistent singleton AudioContext architecture:

- Singleton Pattern: AudioContext is instantiated once at root level and preserved across React page navigation.
- Explicit Stream Separation: Video and audio streams are separated to prevent webcams overriding headset mics.
- Linear PCM Chunks: Downsamples microphone audio to 16kHz mono Linear PCM for zero-latency STT ingestion.

2.2 Waiting Room & State Rehydration

Upon completing the 6-stage check, candidates enter WaitingRoom.jsx featuring a synchronized countdown timer aligned to start_date and start_time. If a candidate accidentally reloads (F5) during an active assessment, interviewSession.js interrogates GetInterviewSessionView to rehydrate the exact question index and previous answers without resetting progress.

3. Test Evaluation & Real-Time Interview Loop

The evaluation engine operates as a concurrent, event-driven loop. Each answer submitted by the candidate is immediately evaluated across three distinct cognitive dimensions while simultaneously generating the next dynamically scaled technical inquiry.

3.1 Rigorous 3-Dimensional Answer Scoring

In groq_service.py, the evaluate_answer() method evaluates the verbatim candidate response against the question text using a supportive yet thorough technical rubric:

Scoring Dimension
Relevance
Accuracy
Clarity
Overall Score

3.2 Parallel Concurrency via ThreadPoolExecutor

To minimize candidate latency between questions, SubmitAnswerView leverages Python's concurrent.futures.ThreadPoolExecutor to execute two intensive operations in parallel:

- Worker Thread A: Executes ai_service.evaluate_answer() to compute 3D scores and qualitative feedback.
- Worker Thread B: Executes ai_service.generate_next_question() to synthesize the subsequent question.
- Turnaround Latency: Slashes total API roundtrip latency from ~6.2 seconds down to ~2.8 seconds.

3.3 Adaptive Question Difficulty Scaling

The AI interviewer dynamically tunes question complexity according to the candidate's demonstrated mastery:

- Previous Score > 85%: Senior Level: Probes distributed trade-offs, concurrency, scaling edge cases.
- Previous Score 60 - 85%: Intermediate Level: Investigates practical tooling decisions, implementation details.
- Previous Score < 60%: Foundational Level: Pivots smoothly ('Let's switch gears...') to core definitions.
- Anti-Duplication Filter: Maintains question_bank in memory; explicitly forbids repeating questions.
- Non-IT Role Boundaries: For non-technical roles, strictly suppresses coding questions; focuses on domain.

Visual metric breakdown across Relevance, Accuracy, and Clarity for all 5 questions

A concise 2-3 sentence overview of candidate problem-solving and communication depth

4. Proctoring, Live Monitoring & Report Synthesis

Explicit list of engineering strengths, design patterns, and competencies proven

4.1 Multi-Layered Proctoring & Malpractice Detection

Identified knowledge gaps, inaccurate answers, or hand-waving detected by AI

Dwaani enforces an active integrity guardrail throughout the entire assessment session to prevent unauthorized collaboration, multi-tab browsing, or generative AI cheating:

- Tab-Switch Detection: Listens to document.visibilitychange events; flags tab transitions or minimized windows.
- Dual-Frame Canvas Blending: Captures webcam & shared screen frames; blends them onto HTML5 canvas.
- Anomaly Logging: Pushes snapshots to LogAnomalyView with severity classification ('low', 'medium', 'high').
- Malpractice Abort Trigger: On excessive violations, status shifts to 'malpractice', forcing score=0 and Reject.

Clear, actionable recommendation: 'Hire', 'Reject', or 'Maybe'

Live USD-to-INR API rate calculating exact token inference cost (INR 10-15 per candidate)

4.2 Recruiter Live Monitoring Command Center

Recruiters can observe live interviews in real-time via LiveMonitoring.jsx connected to Django Channels WebSocket endpoint ws/interview/<session_token>/. Live events include:

- Live Q&A Stream: Broadcasts each question and candidate spoken answer in real time.
- Instant 3D Telemetry: Displays live Relevance, Accuracy, and Clarity score dials upon answer submission.
- Proctoring Alerts: Displays real-time anomaly snapshots whenever focus changes or tab-switches occur.

4.3 Final Report Compilation & INR Cost Telemetry

Upon session completion, generate_interview_report() compiles the comprehensive hiring scorecard:

Report Section
Candidate Profile
3D Radar Breakdown
Executive Summary
Verified Strengths
Vulnerabilities
Hiring Recommendation
INR Financial Telemetry

5. Application Architecture & Component Structure

The Dwaani platform is engineered with a clean, decoupled full-stack architecture separating backend micro-services from the unified React presentation layer.

5.1 Backend Django 5.0 Apps & Model Mapping

Module / File
models/interviews_models.py
models/reports_models.py
services/groq_service.py
views/interviews_views.py
views/screening_views.py
websockets/consumers.py
websockets/stt_consumer.py

5.2 Frontend React 19 Unified Portal Mapping

Component File
pages/recruiter/Jobs.jsx
pages/recruiter/LiveMonitoring.jsx
pages/recruiter/ReportDetail.jsx
pages/candidate/SystemCheck.jsx
pages/candidate/WaitingRoom.jsx
pages/candidate/Interview.jsx
pages/candidate/Feedback.jsx

BLUEPRINT VERIFICATION & REPOSITORY SUMMARY

x95 End-to-end architecture verified against production repository codebase.
x95 Zero mock placeholders: all endpoints, models, prompt structures, and components are fully implemented.
x95 Standalone Mermaid architecture diagram available in DWANI_COMPLETE_FLOW.mmd.
x95 Full markdown specification document available in DWANI_FLOW_DOCUMENTATION.md.